

Revision Question Alternative

Question 1

- a) Write a query to calculate the total revenue generated by each product across all transactions.
- Use the SUM() window function partitioned by product_id.
 - Join products and transaction_items to link names with sales.
 - Output: product_id, product_name, and the total revenue per product (as product_revenue).
- b) Write a query to identify all transactions where the total_amount is lower than the overall average transaction amount.
- Use a subquery to calculate the average total_amount from the transactions table
 - Output: transaction_id, transaction_date, and total_amount
- c) Write a query to list every item sold, including the customer's name and the date it was bought.
- Use JOINS to connect customers, transactions, and transaction_items.
 - Output: first_name, last_name, transaction_date, and product_id.

d) Categorize your customers based on the number of transactions they have made.

- Use a CASE statement to label customers as:
- 'Frequent' for customers with more than 3 transactions.
- 'Regular' for customers with 1 to 3 transactions.
- 'New' for customers with 0 transactions.
- Output: customer_id, last_name, and loyalty_tier.

e) Calculate the running total of stock quantities within each product category.

- Use the SUM() window function.
- Partition by category and order by product_name.
- Output: category, product_name, stock_quantity, and category_running_stock.

f) Find products that are more expensive than the average price within their specific category.

- Use the AVG() window function partitioned by category.
- Wrap this in a subquery or CTE to filter only those where price > avg_category_price.
- Output: product_name, category, price, and avg_category_price.